

Chess Master

Object-Oriented Chess Game with SFML Graphics

Project Report

Course: Object-Oriented Programming | Language: C++

Graphics Library: SFML (Simple and Fast Multimedia Library)

Submitted by: **Moaz Shafie**

Student ID: **25F-0029**

Submitted to: **Hassan Sami**

Date: **14 May 2026**

TABLE OF CONTENTS

1	Introduction	3
2	Objectives	3
3	Tools and Technologies	3
4	Object-Oriented Design	4
4.1	Class Hierarchy — Piece and Subclasses	4
4.2	Board Class	4
4.3	Game Class	5
4.4	Main Entry Point	5
4.5	Supporting Code	5
5	Game Rules Implementation	5
6	SFML User Interface	6
7	Project File Structure	6
8	Design Decisions and Rationale	7
9	Testing and Verification	7
10	Screenshots	8
11	UML Class Diagram	8
12	Limitations and Possible Extensions	9
13	Conclusion	9
14	Student Declaration	9

★ Abstract

This project implements a two-player chess game with optional play against the computer, using **C++** and **object-oriented design**. The game enforces standard chess movement for all pieces, including special moves such as castling and en passant, and detects check, checkmate, and stalemate. A graphical user interface is built with **SFML** (Simple and Fast Multimedia Library): a main menu, an 8×8 board with textured pieces, a side panel for status messages, and mouse-driven piece selection and movement. The codebase separates declaration (*Header.h*), implementation (*Implementation.cpp*), and program entry (*Main.cpp*) to follow modular C++ practice.

1. Introduction

Chess is a suitable domain for an OOP assignment because each piece type shares common behaviour (colour, whether it has moved, validity testing) while differing in movement rules. This project models that relationship with a base **Piece** class and derived classes per piece type. SFML is used for cross-cutting presentation: window management, textures, sprites, text, shapes, and input events.

The project demonstrates how inheritance and polymorphism remove the need for repetitive switch-on-type logic in a real, interactive application, and how clean separation between game logic and rendering leads to maintainable code.

2. Objectives

- Model chess pieces using **inheritance and polymorphism** (virtual methods).
- Represent the board as a 2-D array of pointers to **Piece**, allowing empty squares (*nullptr*) and dynamic piece types.
- Implement complete move validation per piece, plus **castling**, **en passant**, and **pawn promotion** (to queen in the current design).
- Implement **check detection** by scanning whether any opponent piece can legally attack the king's square.
- Prevent moves that leave the current player's king in check (illegal move rejection).
- Detect **checkmate** and **stalemate** via *HasAnyValidMove* for the side to move.
- Provide an **SFML GUI**: mode selection, board rendering, highlights, fonts, and status text.
- Optional: simple AI for black in 1-player mode (random choice among all legal moves).

3. Tools and Technologies

Component	Detail
Language	C++ (ISO C++17)
Graphics / Window / Input	SFML — sf::RenderWindow, sf::Texture, sf::Sprite, sf::Font, sf::Text, sf::RectangleShape, mouse events

Standard Library	string, iostream, cstdlib, ctime, map
Build System	Visual Studio project (.vcxproj) linking against SFML libraries
AI	Random move selection (legal moves only, seeded with ctime)

4. Object-Oriented Design

4.1 Class Hierarchy — Piece and Subclasses

The **Piece** class is the abstract base for every chess piece. It encapsulates common state (colour char, *hasMoved* flag) behind public getters/setters and declares two pure-virtual interface methods:

```
virtual bool IsValidMove(int sx, int sy, int dx, int dy, Piece* Grid[8][8]) = 0;
virtual char GetSymbol() const = 0;
```

Six concrete subclasses inherit from **Piece**: **Pawn**, **Rook**, **Knight**, **Bishop**, **Queen**, **King**. Each overrides the above methods with its unique movement logic. **Queen** reuses existing logic by delegating to temporary **Rook** and **Bishop** objects — a clean example of *composition of rule checks* without multiple inheritance.

Class	Symbol	Key Rule / Override
Pawn	P / p	Forward 1, initial 2-step, diagonal capture, en passant
Rook	R / r	Horizontal & vertical sliding; castling partner
Knight	N / n	L-shaped jump; ignores blocking pieces
Bishop	B / b	Diagonal sliding
Queen	Q / q	Delegates to Rook + Bishop rules
King	K / k	One square any direction; castling if unmoved & path clear

4.2 Board Class

The **Board** class owns the 8×8 grid of *Piece** pointers and carries all game-state transitions:

- **Setup()** — places initial pieces on both sides.
- **Move(sx, sy, dx, dy, Turn)** — validates turn, applies special-move handling, simulates position to reject moves into check, deletes captured pieces, updates *hasMoved*, records last move for en passant, calls **PromotePawn** on last rank.
- **IsCheck(Colour)** — finds the king, tests whether any enemy *IsValidMove* can reach that square.
- **HasAnyValidMove(Colour)** — brute-forces all origin/destination squares with undo to verify that at least one move leaves the king safe.
- **Display()** — ASCII board to console (debugging aid).
- **Destructor** — deletes all Piece objects to prevent memory leaks.

4.3 Game Class

The **Game** class glues together logic and presentation. It holds a **Board** instance by composition and manages all SFML resources:

- **LoadAssets()** — loads font and piece PNG paths into a texture map keyed by symbol char.
- **HandleEvents()** — processes window close, menu clicks, and board clicks for select / deselect / move when it is the human's turn.
- **Render()** — draws the board, highlights, piece sprites or fallback text, and the side status panel.
- **RenderMenu()** — draws the title and two mode-selection buttons.
- **MakeComputerMove()** — collects all legal moves for black, picks one at random, calls **Move**, and updates status messages.
- **Start()** — main loop: poll events → branch on state → AI step if needed → render.

4.4 Main Entry Point

main() in *Main.cpp* instantiates **Game** and calls **Start()** — deliberately minimal to separate concerns.

4.5 Supporting Code

A legacy **convert()** function translates algebraic notation strings (e.g. "e2") to board-index pairs. It is retained for compatibility even though the GUI uses direct mouse-coordinate mapping.

5. Game Rules Implementation

Piece / Rule	Implementation Detail
Pawn	Forward 1 square; initial 2-square push; diagonal capture only; en passant triggered when last move was a double push of adjacent pawn.
Rook	Horizontal & vertical sliding; blocked by intervening pieces.
Bishop	Diagonal sliding; blocked by intervening pieces.
Knight	L-shaped jump (2+1 squares); not blocked by pieces in between.
Queen	Combined Rook + Bishop movement via delegation.
King	One square in any direction; castling when king & rook unmoved, path clear
Check	After every move, IsCheck() verifies the moving side's king is safe.
Checkmate	IsCheck() is true AND HasAnyValidMove() returns false.
Stalemate	IsCheck() is false AND HasAnyValidMove() returns false.
Promotion	Pawn reaching the back rank is automatically replaced with a Queen.

6. SFML User Interface

Window & Layout

- **Window:** 1000×800 px, titled "Chess Master - SFML", frame rate capped at 60 FPS.

- **Board region:** 800×800 px — 8 rows × 8 columns of 100×100 px cells.
- **Side panel:** right-hand 200 px strip for title, turn indicator, and status messages.

Visual Design

- Alternating light/dark *wood-tone* cell colours for the board.
- Selected square highlighted in yellow; last-move squares optionally tinted.
- Piece PNG assets loaded from an `assets/` folder; fallback to large **sf::Text** symbol if a texture fails to load.
- Status messages coloured red for warnings/check, green otherwise.

Input Handling

- Mouse click on board → compute cell ($x \div 100$, $y \div 100$) → first click selects piece, second click executes or re-selects.
- Menu buttons detected by pixel-rectangle hit-test on **sf::Event::MouseButtonReleased**.
- Invalid moves silently deselect and display an error string in the panel.

7. Project File Structure

File / Folder	Purpose
Header.h	All class/enum declarations; <code>convert()</code> prototype
Implementation.cpp	All method definitions: pieces, board, game loop, SFML rendering
Main.cpp	<code>main()</code> — single call to <code>Game::Start()</code>
assets/	Fonts (.ttf) and piece images (.png) referenced at runtime
SFML SDK / .vcxproj	Linked libraries and include paths configured in Visual Studio

8. Design Decisions and Rationale

- **Pointer-based board:** allows polymorphic *Piece** in each cell and simple capture (replace/delete) without object slicing.
- **Virtual functions:** Board and check-detection code call *IsValidMove* without any switch-on-type logic for the general case — clean OOP practice.
- **dynamic_cast:** used only where special rules depend on concrete type (Pawn, King) — justified for clarity in a course project.
- **Brute-force move search:** simple and correct for 8×8; performance is acceptable for a learning project and avoids premature optimisation.
- **Queen delegation:** avoids duplicating sliding logic; demonstrates composition inside the class hierarchy.
- **Three-file layout:** separates interface (Header.h), implementation, and entry point — follows conventional C++ project structure.

9. Testing and Verification

Test Case	Expected Result	Status
Starting position setup	All 32 pieces placed correctly	✓ Pass
Pawn forward 1 / initial 2	Move accepted	✓ Pass
Pawn illegal sideways move	Move rejected	✓ Pass
Rook/Bishop blocked by own piece	Move rejected	✓ Pass
Knight jumps over pieces	Move accepted	✓ Pass
Castling (kingside & queenside)	King+Rook swap correctly	✓ Pass
Castling through check — blocked	Move rejected	✓ Pass
En passant immediately after push	Capture executed correctly	✓ Pass
En passant one move later	Move rejected	✓ Pass
Pawn promotion to queen	Pawn replaced with Queen	✓ Pass
Move leaving own king in check	Move rejected; message shown	✓ Pass
Checkmate detection	"Checkmate" message; game ends	✓ Pass
Stalemate detection	"Stalemate" message; game ends	✓ Pass
2-Player alternation	Turns alternate correctly	✓ Pass
1-Player: computer auto-moves	Black plays random legal move	✓ Pass
Missing asset — fallback text	Symbol char rendered instead	✓ Pass

10. Screenshots

Replace the placeholder boxes below with actual screenshots when assembling the final document.

[Screenshot 1: Main Menu]

Shows the two mode-selection buttons (1 Player vs Computer / 2 Players Local) on the dark-themed start screen.

**[Screenshot 2:
Mid-Game Board]**

Mid-game state showing white and black pieces, a selected square highlighted in yellow, and the side panel with current turn and status message.

**[Screenshot 3:
Checkmate Screen]**

Final position with the Checkmate message displayed in the side panel, confirming correct endgame detection.

11. UML Class Diagram

```
+-----+ +-----+ | <<abstract>> | | Board | | Piece |
|-----| |-----| | -Grid[8][8]:Piece* | | #color: char | |
-lastMove: struct | | #hasMoved: bool | |-----| |-----| |
+Setup() | | +GetColor(): char | owns | +Move(...) | | +IsValidMove(...) |<-----|
+IsCheck(char) | | +GetSymbol(): char | 0..* | +HasAnyValidMove() |
+-----+ +-----+ | +PromotePawn(...) | | +-----+ +-----+ |_____|_____|
composition | | | | Pawn Rook ... King +-----+ +-----+ | Game |
|-----| | -B: Board | | -Turn: char | | -window: RWin | |-----|
| +Start() | | +Render() | | +HandleEvents() | | +MakeAIMove() | +-----+
```

Figure 1: Simplified UML class diagram

12. Limitations and Possible Extensions

Current Limitations

- **AI strength:** random legal-move selection only — no search depth, opening book, or material evaluation.
- **Promotion:** always promotes to Queen; no under-promotion dialog.
- **No persistence:** no replay save, PGN export, or undo stack.
- **No clock:** chess timer not implemented.

Possible Extensions

Extension	Description
Minimax / Alpha-Beta AI	Replace random AI with a depth-limited search for stronger play.
Promotion dialog	Pop-up to choose Queen / Rook / Bishop / Knight on promotion.
Move history list	Scrollable algebraic notation panel alongside the board.
Chess clock	Per-player countdown with forfeit on time expiry.
Sound effects	SFML audio for move, capture, check, and checkmate events.
PGN export	Save completed games in standard Portable Game Notation.
Online multiplayer	TCP socket layer so two players can connect over a network.

13. Conclusion

The project successfully demonstrates core OOP principles — **encapsulation**, **inheritance**, and **polymorphism** — within a complete, interactive chess application. The abstract **Piece** hierarchy eliminates repetitive type-checking code, and the **Board/Game** separation cleanly divides game logic from presentation — a design pattern directly applicable to professional game development.

SFML proved an effective choice for a course project: its straightforward API allowed a polished GUI without the overhead of a full game engine, while keeping focus on the C++ design. The brute-force move-legality approach is entirely adequate for an 8×8 board and makes the code easy to follow and verify.

Possible next steps — such as a minimax AI, promotion dialog, and move history — would extend the project into a more feature-complete application without requiring fundamental architectural changes, demonstrating the value of the modular design chosen.

14. Student Declaration

I declare that this report and the submitted source code represent my own work except where explicitly cited. All external references (SFML documentation, standard chess rules) have been acknowledged appropriately. I understand the penalties for academic misconduct as defined by my institution's policy.

Full Name: **Moaz Shafie** Student ID: **25F-0029**

Submitted to: **Hassan Sami** Date: **14 May 2026**